Rapport des TP – Séances 5 et 6

React avec Vite : hooks, état local et état global

Firas Farhat • Groupe : .....
Module MERN Stack – Dr. Abdelweheb GUEDDES – École Polytechnique de Sousse – 2026/2027
Dépôt GitHub : https://github.com/firasfarhat10/mon-app-state

Introduction

Après quatre séances consacrées au back-end (Express, MongoDB, JWT), les séances 5 et 6 ont été l'occasion de passer au front-end avec React. La séance 5 portait sur les bases : création d'un projet avec Vite, syntaxe JSX, composants et les deux hooks principaux, `useState` et `useEffect`, à travers un compteur interactif. La séance 6 traitait de l'état global : le partage d'un thème clair / sombre avec `useContext`, puis un panier de commerce électronique géré avec Redux Toolkit.

Les deux applications ont été validées en séance, avec leurs défis. Ce rapport présente ce que j'ai réalisé, les choix techniques et les problèmes rencontrés. Le code de l'application de la séance 6 est publié sur GitHub.

Élément	Version utilisée
Node.js	24
Vite	8
React / React DOM	19
Redux Toolkit / React Redux (séance 6)	2.14 / 9.3

Séance 5 – React, JSX et hooks fondamentaux

1. Création du projet et fonctionnement de Vite

Le projet a été créé avec `npm create vite@latest mon-compteur-react -- --template react`, puis `npm install` et `npm run dev`. Vite démarre presque instantanément et met à jour la page à chaque enregistrement sans la recharger (HMR).

Réponse au « Prédisez » : la page `index.html` ne contient qu'un `<div id="root"></div>` vide. Tout ce qui s'affiche est créé par JavaScript : `index.html` charge `src/main.jsx`, qui appelle `createRoot(document.getElementById('root')).render(<App />)`. React exécute ensuite le composant App et les composants qu'il contient, et construit lui-même les éléments HTML. On le vérifie facilement : « Afficher le code source » montre le div vide, alors que l'onglet Éléments des outils de développement montre toute l'interface générée.

2. Premier composant : Header

Un composant React est une fonction JavaScript qui retourne du JSX. `Header` reçoit deux props (`titre` et `sousTitre`), calcule la date du jour en français avec `toLocaleDateString('fr-FR', ...)`, puis retourne le titre, le sous-titre et la date. J'ai appliqué les règles du JSX vues en cours : un seul élément racine, `className` au lieu de `class`, balises fermées, expressions JavaScript entre accolades et `style` passé sous forme d'objet.

3. useState : le compteur

Réponse au « Prédisez » (compteur raté) : avec une variable `let compteur = 0`, la console affiche bien 1, 2, 3, mais l'écran reste à 0. Modifier une variable ordinaire ne prévient pas React, qui ne relance donc pas le rendu ; et même s'il le faisait, la ligne `let compteur = 0` remettrait la valeur à zéro. `useState` règle les deux problèmes : la valeur est conservée par React entre les rendus, et l'appel du setter déclenche un nouveau rendu.

J'ai ensuite enrichi le compteur : un pas réglable (1, 2, 5 ou 10) avec un `<select>`, un message de dernière action, le total des clics, un historique et une couleur selon le signe de la valeur. Deux points m'ont demandé de l'attention :

- `e.target.value` renvoie toujours une chaîne. Sans `parseInt`, le calcul `0 + "5"` donne `"05"` par concaténation.
- L'historique est un tableau : il ne faut jamais le modifier avec `push`. React compare les références, et un même tableau modifié est considéré comme inchangé. J'utilise donc `setHistorique([...historique, action])`, qui crée un nouveau tableau.

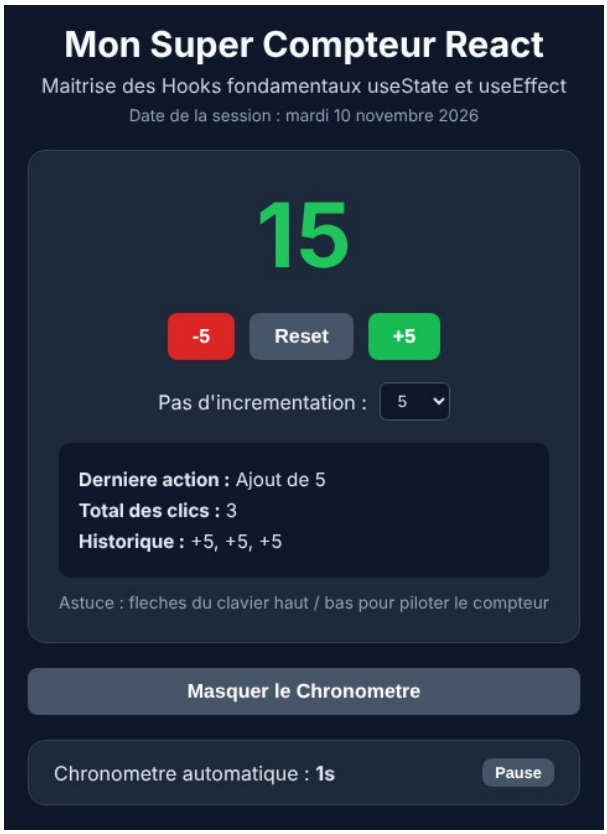


Figure 1 – Pas de 5, trois clics sur +5 : compteur à 15 en vert, total 3, historique +5, +5, +5.

4. useEffect : titre, localStorage et chronomètre

Deux effets dépendent de `count` (tableau de dépendances `[count]`) : l'un met à jour le titre de l'onglet (« Compteur : 15 » sur la capture), l'autre sauvegarde la valeur dans le `localStorage`. Au démarrage, la valeur est relue grâce à une initialisation paresseuse : `useState(() => ...)`.

Réponse au « Prédisez » (42 puis F5) : le compteur ne revient pas à zéro. L'effet a écrit la valeur dans le `localStorage` à chaque changement, et l'initialisation de `useState` la relit au rechargement. Sans `localStorage`, l'état React vit en mémoire et disparaît au F5, comme les tableaux du serveur Express en séance 2.

Le chronomètre (`AutoTimer`) lance un `setInterval` dans un effet qui dépend de `isRunning` et retourne une fonction de nettoyage `clearInterval`. Quand on clique sur « Masquer le Chronomètre », le composant est démonté : React appelle le nettoyage et le timer s'arrête. Quand on le réaffiche, il repart de 0, car un composant démonté perd son état. Sans ce nettoyage, en mode développement, le chronomètre avance de 2 en 2, car React StrictMode monte, démonte puis remonte chaque composant pour révéler justement les nettoyages oubliés.

5. Défis de la séance 5

Défi 1 – Bornes ±50. Toutes les modifications passent par une seule fonction `changer(delta)`, qui calcule la nouvelle valeur avant de l'appliquer. Si elle dépasse une borne, l'état n'est pas modifié et le message « Seuil maximal de 50 atteint ! » s'affiche en orange. Le bouton concerné est aussi désactivé avec l'attribut `disabled` quand le prochain pas dépasserait la borne. Les deux protections sont utiles : `disabled` ne bloque que la souris, alors que le clavier passe aussi par `changer`.

```
Counter.jsx (extrait)
const changer = (delta) => {
  const nouveauCompte = count + delta;
```

```
  if (nouveauCompte > MAX) {
    setMessage(`Seuil maximal de ${MAX} atteint !`);
    setAlerte(true);
    return;
  }
  // ... même test pour MIN
  setCount(nouveauCompte);
  setHistorique([...historique, (delta > 0 ? '+' : '') + delta]);
};
```

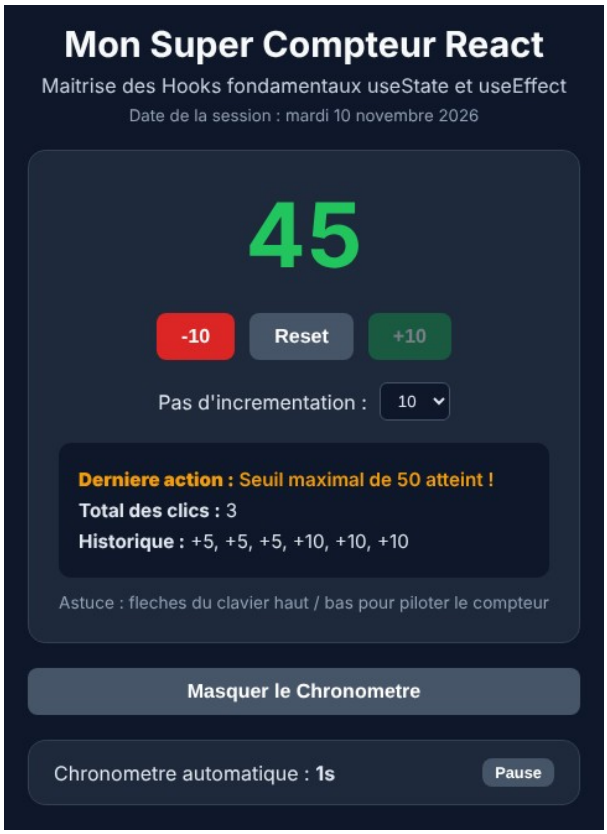


Figure 2 – Pas de 10 : bloqué à 45, bouton +10 désactivé et message d'alerte.

Défi 2 – Historique persistant. Le tableau est converti en texte avec `JSON.stringify` dans un effet qui dépend de `historique`, puis relu avec `JSON.parse` dans l'initialisation de `useState`, protégée par un `try/catch` au cas où la valeur stockée serait corrompue. Restaurer l'historique dans un `useEffect(..., [])` ne fonctionne pas si l'effet de sauvegarde est écrit avant : React l'exécute en premier, il écrit `[]` dans le `localStorage`, et la restauration relit ce tableau vide. La lecture dans l'initialisation de `useState` a lieu pendant le premier rendu, avant tout effet, ce qui règle le problème.

Défi 3 – Contrôle au clavier. L'écouteur `keydown` est attaché à `window` dans un effet, et retiré dans la fonction de nettoyage avec `removeEventListener`. Avec un tableau de dépendances vide `[]`, la fonction garderait les valeurs du premier rendu (closure périmée) : le pas choisi serait ignoré et le compteur resterait bloqué. En mettant les fonctions d'incrémentation dans les dépendances, l'effet retire l'ancien écouteur et en pose un nouveau à chaque rendu, qui voit toujours les valeurs à jour. J'ai aussi ignoré les flèches quand le focus est sur le `<select>`, sinon une même touche modifiait le pas et le compteur.

```
Counter.jsx (défi 3)
useEffect(() => {
  const handleKeyDown = (e) => {
```

```
    if (e.target.tagName === 'SELECT') return;
    if (e.key === 'ArrowUp') { e.preventDefault(); handleIncrement(); }
    else if (e.key === 'ArrowDown') { e.preventDefault(); handleDecrement(); }
  };
  window.addEventListener('keydown', handleKeyDown);
  return () => window.removeEventListener('keydown', handleKeyDown); // nettoyage
}, [handleIncrement, handleDecrement]);
```

Séance 6 – État global : useContext et Redux Toolkit

1. Le problème du Prop Drilling

Quand une donnée doit traverser plusieurs composants intermédiaires qui ne l'utilisent pas, uniquement pour atteindre un composant profond, le code devient rigide et difficile à maintenir : c'est le prop drilling.

Réponse au « Prédisez » : si un composant d'une autre branche a aussi besoin du panier, il faut remonter l'état avec `useState` jusqu'au plus proche ancêtre commun, souvent App, puis le redescendre par les props dans les deux branches. Conséquence : à chaque modification du panier, App se re-rend, et avec lui tout son sous-arbre, y compris les composants qui n'utilisent pas le panier.

2. Le thème avec useContext

Le fichier `ThemeContext.jsx` suit les trois temps vus en cours : `createContext` crée le contexte, le composant `ThemeProvider` détient l'état `theme` et le diffuse avec `<ThemeContext.Provider value={{ theme, toggleTheme }}>`, et le hook personnalisé `useTheme()` permet à n'importe quel descendant de le lire. Ce hook lève une erreur explicite s'il est utilisé en dehors du Provider. Le thème est sauvegardé dans le `localStorage`.

Avec le code de l'énoncé, cliquer sur « Mode Sombre » changeait bien la valeur de `theme`, mais rien ne changeait à l'écran : aucune instruction n'appliquait le thème à la page. J'ai ajouté une ligne dans l'effet du Provider, `document.documentElement.dataset.theme = theme`, et défini les couleurs du CSS avec des variables, redéfinies pour `[data-theme='dark']`. Le changement de thème met ainsi à jour toute la page, sans passer de prop à aucun composant.

3. Redux Toolkit : store, slice et flux de données

Le panier est géré avec Redux Toolkit (`npm install @reduxjs/toolkit react-redux`). L'organisation suit les trois principes de Redux : un **store unique** (`store.js` avec `configureStore`), un état en **lecture seule** qu'on ne modifie qu'en envoyant des actions (`dispatch`), et des **reducers** qui calculent le nouvel état. Le slice `cartSlice.js` est créé avec `createSlice`, qui génère automatiquement les actions (`cart/addItem`, `cart/removeItem`, `cart/clearCart`…).

Dans les reducers, on peut écrire `existing.quantity += 1` ou `state.items.push(...)` : cela ressemble à une mutation, mais Redux Toolkit utilise la bibliothèque Immer, qui travaille sur un brouillon et produit un nouvel état immuable. Les totaux sont des valeurs dérivées : une fonction `recalculateTotals` les recalcule à partir des articles après chaque action, ce qui évite toute incohérence.

Dans `main.jsx`, l'application est enveloppée par `<Provider store={store}>` puis par `<ThemeProvider>`. `App.jsx` affiche la barre de navigation, le catalogue et le panier sans leur passer aucune prop : chaque composant lit lui-même ce dont il a besoin.

Composant	État lu	Rôle
Navbar	`useTheme()` + `useSelector(state => state.cart.totalCount)`	Bouton de thème et badge du panier
ProductList / ProductCard	`useSelector(selectQuantityById(id))`	Catalogue, `dispatch(addItem(produit))`
Cart	`useSelector(state =>`	Lignes, quantités +/-, vidage,

Composant	État lu	Rôle
	`state.cart)`	code promo, totaux

La Navbar ne sélectionne que `totalCount` : elle ne se re-rend que si ce nombre change. C'est l'avantage principal de Redux sur le Context pour des données qui changent souvent : avec un Context, tous les composants consommateurs se re-rendent à chaque modification. L'extension Redux DevTools, activée automatiquement par `configureStore`, permet de suivre chaque action envoyée et l'état qui en résulte.

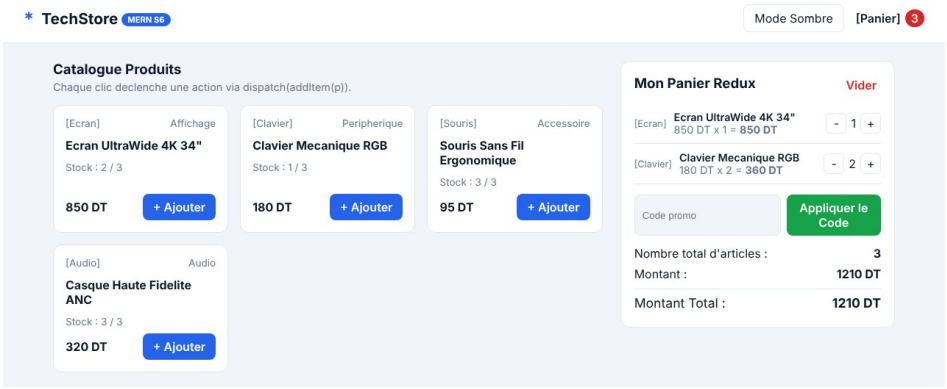


Figure 3 – Un écran et deux claviers : badge à 3, total de 1210 DT.

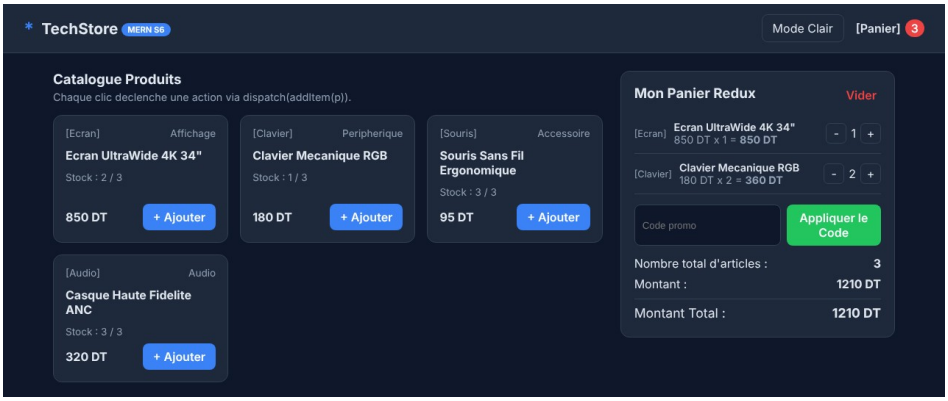


Figure 4 – Mode sombre via useContext : le panier Redux n'est pas affecté.

4. Défis de la séance 6

Défi 1 – Panier persistant. Le tableau `items` est relu dans le `localStorage` (clé `techstore_cart`) pour construire l'état initial, et sauvegardé après chaque action. Restaurer seulement `items` ne suffit pas : après F5, les articles reviennent mais le badge affiche 0, car les totaux de l'état initial restent à zéro. J'appelle donc `recalculateTotals(initialState)` juste après la création de l'état initial. La lecture est protégée par un `try/catch` : une valeur corrompue donne un panier vide au lieu d'un écran blanc.

Je note une contradiction avec la théorie : un reducer doit être une fonction pure, alors qu'écrire dans le `localStorage` est un effet de bord. J'ai suivi la consigne de l'énoncé, mais une solution plus propre serait de sauvegarder depuis `store.subscribe(...)` dans `store.js`, ce qui garderait les reducers purs.

Défi 2 – Code promo. L'état contient `discountPercent`, et un nouveau reducer `applyPromoCode` accepte le code `POLY2027` sans tenir compte des majuscules ni des espaces (`trim().toUpperCase()`). La remise et le prix final sont calculés dans `recalculateTotals` : la réduction reste donc juste si on ajoute ou retire des articles après avoir saisi le code. Un code invalide

affiche « Code promo invalide ». Le texte saisi est un état local (`useState`) du composant `Cart`, car il ne concerne que lui.

```
cartSlice.js (défi 2)
applyPromoCode: (state, action) => {
  const code = action.payload.trim().toUpperCase();
  if (PROMO_CODES[code]) {
    state.discountPercent = PROMO_CODES[code];   // 20 pour POLY2027
    state.promoCode = code;
    state.promoError = '';
  } else {
    state.discountPercent = 0;
    state.promoError = 'Code promo invalide';
  }
  recalculateTotals(state); // remise et prix final recalculés
}
```

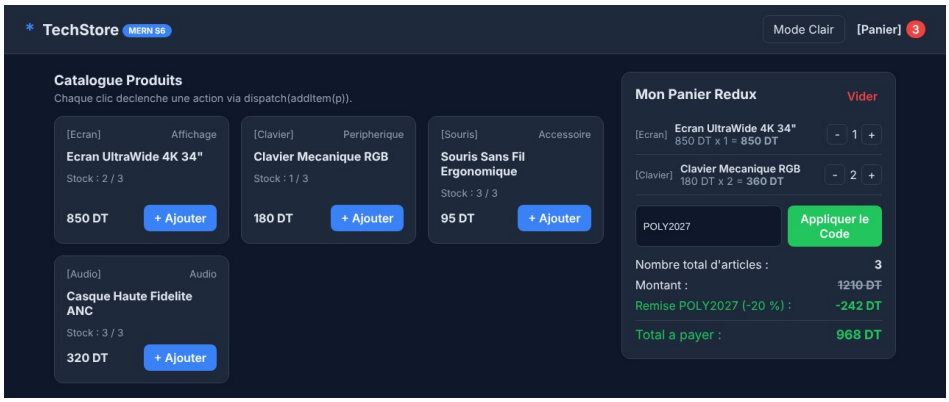


Figure 5 – Code POLY2027 : 1210 DT barré, remise de 242 DT, total à payer 968 DT.

Défi 3 – Stock. Chaque produit a un champ `stock: 3`. J'ai extrait la carte produit dans un composant `ProductCard`, qui lit la quantité de son produit dans le store grâce au sélecteur `selectQuantityById(id)`, et affiche « Rupture de Stock » avec un bouton désactivé quand le stock est atteint. Le bouton + du panier permettait encore de dépasser le stock : la vérification est donc faite aussi dans le reducer `addItem`. La règle métier est dans le slice, et aucun bouton ne peut la contourner.

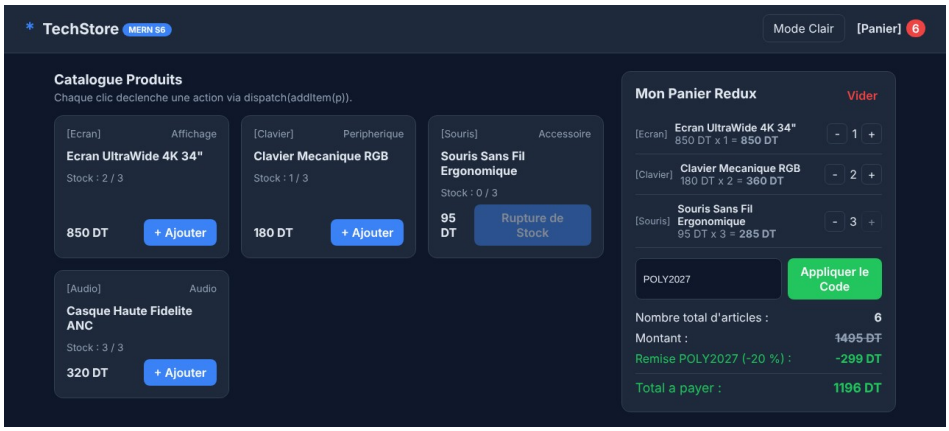


Figure 6 – Trois souris dans le panier : stock 0 / 3, bouton « Rupture de Stock » désactivé.

Comparaison et bilan

Critère	useContext	Redux Toolkit
Installation	Aucune, natif à React	2 dépendances
Re-rendus	Tous les consommateurs du contexte	Seuls les composants dont la donnée sélectionnée a changé
Logique métier	Dans les composants	Isolée dans le slice (reducers)
Débogage	Console	Redux DevTools (historique des actions)
Utilisation dans le TP	Thème clair / sombre	Panier, remise, stock

Ces deux séances m'ont fait passer de l'état local d'un composant à l'état partagé de toute une application. Pour la séance 5, je retiens surtout le fonctionnement du rendu : React ne réagit qu'aux changements d'état faits avec un setter et à de nouvelles références, et les effets doivent toujours nettoyer ce qu'ils créent. Pour la séance 6, la règle est de choisir l'outil selon la donnée : le Context pour une valeur légère qui change rarement, Redux Toolkit pour des données métier qui changent souvent et ont des règles (totaux, remise, stock).

Ces notions me serviront directement dans le projet de fin de module, où l'utilisateur connecté et les données de l'API devront être partagés entre plusieurs pages.

Dépôt GitHub

Le projet de la séance 6 est publié dans le dépôt `mon-app-state` : https://github.com/firasfarhat10/mon-app-state

```
Organisation du dépôt
mon-app-state/
├── README.md
├── .gitignore              (node_modules et dist exclus)
├── index.html
├── package.json
└── src/
    ├── main.jsx            Provider Redux + ThemeProvider
    ├── App.jsx
    ├── index.css           thèmes clair et sombre (variables CSS)
    ├── context/ThemeContext.jsx
    ├── redux/store.js
    ├── redux/cartSlice.js
    └── components/         Navbar, ProductList, ProductCard, Cart
```

Pour lancer le projet : `npm install` puis `npm run dev`, et ouvrir http://localhost:5173. Le code promo de test est `POLY2027`.